# ParentBot: Complete Build Roadmap (Go)

*A solo, learning-by-doing plan for a WhatsApp voice assistant that helps parents book cabs and food in Hindi/Hinglish.*

## 1. Project overview

**What it is:** A WhatsApp AI agent for parents. A parent sends a voice note like 'mere liye cab book karo'. The bot understands it, asks short follow-up questions, confirms the details, and completes the booking (or sends a one-tap booking link). The child sets up saved places, spending limits and monitors activity through a web dashboard.

**Why it is worth building:** Parents already live on WhatsApp but struggle with Uber, Zomato and BookMyShow (new apps, OTPs, English UI). The person who feels the pain, the child, is the natural customer.

**Your goals as a solo student:**

- Learn Go properly by building something real (you already know Python, so this is your second language).
- Practise production-grade engineering: idempotency, state machines, safety rules in code, testing, observability, CI/CD.
- End with a portfolio project that runs for real with your own family.

**Honest scope:** Production-grade engineering, not a licensed company. You will use the WhatsApp Cloud API test number (up to 5 verified recipient numbers), start with cash or pay-on-ride, and begin with deep links before attempting real network booking.

## 2. Scope: what is in and what is out

**In scope (v1):**

- WhatsApp voice and text input in one language (your parents' language first)
- Speech-to-text with a measured accuracy benchmark
- Conversation engine built as a state machine
- Cab flow in one city (deep link first, ONDC/Beckn sandbox second)
- Saved places, spend caps, confirmation before every booking
- Child dashboard (web)

- Audit log, tests, CI/CD, monitoring, deployment

**Out of scope (later versions):**

- Public launch and Meta business verification
- Becoming a certified ONDC Buyer App
- Handling money yourself (always use the provider's payment or cash)
- Movie booking (BookMyShow has no open path)
- Multiple cities and many languages

**Reality check on supply:** Uber, Zomato and BookMyShow do not offer public booking APIs for third parties. That is why the plan uses deep links first, then the open ONDC/Beckn network. Avoid scraping or automating their apps; it is brittle and against their terms.

## 3. Tech stack

| Layer | Choice | Why |
|---|---|---|
| Backend language | Go 1.22+ | Great for webhooks, workers, concurrency; one static binary |
| HTTP router | chi | Small, idiomatic, close to net/http |
| Database | Postgres (Neon or Supabase free tier) | Correctness, constraints, transactions |
| DB access | pgx + sqlc | You write SQL, sqlc generates type-safe Go |
| Migrations | goose | Simple, SQL-based |
| Queue and jobs | Redis + asynq | Retries, scheduling, backoff |
| Speech-to-text | Sarvam AI, Whisper, Google STT (benchmarked) | Compare on real recordings |
| LLM | Claude via anthropic-sdk-go with tool use | Structured extraction, Hinglish replies |
| Validation | go-playground/validator | Reject malformed LLM output |
| Config | caarlos0/env | 12-factor config |

| Layer | Choice | Why |
|---|---|---|
| Logging | log/slog | Structured logs in the standard library |
| Testing | testing, testify, testcontainers-go, httptest | Real Postgres in tests, fake HTTP servers |
| Auth (dashboard) | Phone OTP + golang-jwt | Familiar to Indian users |
| Dashboard | Next.js, TypeScript, Tailwind, shadcn/ui | Standard, good for placements |
| Observability | sentry-go, Prometheus, Grafana Cloud, UptimeRobot | All have free tiers |
| Security scanning | govulncheck, gosec, Dependabot | Catch known issues early |
| Deploy | Docker, Fly.io or Railway, Cloudflare | Cheap and simple |
| CI/CD | GitHub Actions | Free for student projects |

**Free resources:** Apply for the GitHub Student Developer Pack and cloud credits (AWS, Azure, DigitalOcean).

**Python side tool:** Keep a small Python script with jiwer for computing word error rate in the ASR benchmark. Everything in the product stays in Go.

## 4. System architecture

**Request flow:**

1. Parent sends a WhatsApp message. Meta calls your webhook.
2. The API verifies the signature, stores the message ID (deduplication) and pushes a job to Redis, then returns 200 immediately.
3. A worker picks up the job. If it is a voice note, it downloads the audio and transcribes it.
4. The conversation engine loads the user's state, asks Claude to extract structured intent, validates it, and moves the state machine.
5. The safety layer checks limits and rules.
6. If a booking is confirmed, the booking provider creates the ride (deep link or Beckn).

7. The bot replies on WhatsApp (text, optionally voice) and writes everything to the audit log.

**Golden rule:** The LLM only extracts intent and phrases replies. Your code decides every action, especially anything involving money.

**Repo layout:**

```
parentbot/
├── cmd/
│   ├── api/          # webhook and dashboard API
│   └── worker/       # background jobs
├── internal/
│   ├── whatsapp/     # Meta client, signature check
│   ├── asr/          # speech-to-text providers
│   ├── llm/          # Claude client, intent extraction
│   ├── conversation/ # state machine
│   ├── booking/      # provider interface, deep links, Beckn
│   ├── safety/       # limits and rules
│   ├── store/        # sqlc-generated DB code
│   └── config/
├── migrations/
├── dashboard/        # Next.js app
├── docs/decisions.md # decision log
└── docker-compose.yml
```

Go's internal/ folder keeps packages private to your module, which enforces clean boundaries.

## 5. Data model

| Table | Purpose | Key columns |
|---|---|---|
| users | Parents and children | id, phone, role, language, created_at |
| family_links | Child-to-parent relationship | child_id, parent_id, status |
| saved_places | Named places like ghar, mandir | user_id, label, address, lat, lng |

| Table | Purpose | Key columns |
|---|---|---|
| conversations | Current state per parent | user_id, state, context (JSON), updated_at |
| processed_messages | Idempotency | wa_message_id (unique), received_at |
| orders | Every booking attempt | id, user_id, type, status, pickup, drop, amount, provider, created_at |
| audit_log | Append-only history | id, actor, action, details, created_at |
| limits | Spend caps and rules | user_id, per_ride_cap, daily_cap, allowed_hours |

**Order states:** created, confirmed, booked, completed, cancelled, failed. Enforce legal transitions in code and with database constraints. Make audit_log append-only (no updates or deletes from the app role).

## 6. The roadmap (about 20 weeks, part-time)

### Phase 0: Learn Go (Weeks 1-2, compress to 1 week if quick)

**What:** Learn the Go-specific differences from Python, not programming basics.

**How:**

- Skim *A Tour of Go*, then work through *Go by Example*.
- Read *Learning Go* by Jon Bodner (written for people from other languages).
- Focus on: structs and composition, interfaces (implicitly satisfied), error returns and errors.Is/As, goroutines and channels (versus asyncio), context.Context, net/http, encoding/json, and the testing package.

**Mini project:** An HTTP server that accepts a JSON POST, validates it, stores it in memory, and has table-driven tests.

**Done when:** You can write, test and run a small Go service without looking up basic syntax.

### Phase 1: Project foundations (Weeks 3-4)

**What:** Tooling that keeps quality high from day one.

**How:**

- Create the GitHub repo, go mod init, and the folder layout above.
- Add golangci-lint, gofmt, and a Makefile (make test, make lint, make run).
- docker-compose with Postgres and Redis.
- Config loading from environment variables with caarlos0/env; a .env.example file; secrets never committed.
- Structured logging with log/slog.
- GitHub Actions: lint, test, build on every push.
- Start docs/decisions.md. Log every major choice and why.

**Done when:** A push to GitHub runs CI green, and docker-compose up gives you a working local database and Redis.

## Phase 2: WhatsApp webhook (Weeks 4-5)

**What:** The endpoint that Meta calls for every incoming message.

**How:**

- Create a Meta developer account and a WhatsApp Cloud API app. Use the free test number and add up to 5 recipient numbers (family and friends).
- Implement the GET verification handshake.
- Implement POST handling: read the raw body, verify the X-Hub-Signature-256 header using HMAC-SHA256 (crypto/hmac) with hmac.Equal for constant-time comparison.
- Insert the WhatsApp message ID into processed_messages with a unique constraint. If it already exists, ignore the message. This prevents duplicate bookings from Meta's retries.
- Enqueue the event to Redis with asynq and return 200 right away.
- Build a small whatsapp package (net/http) for sending text replies and fetching media. There is no official Go SDK, and the API is simple.
- Use ngrok or Cloudflare Tunnel for local testing.

**Tests:** Valid signature, invalid signature, duplicate message, malformed JSON.

**Done when:** You send a WhatsApp message and see one verified, deduplicated log line, even if Meta sends it twice.

## Phase 3: Voice pipeline (Weeks 5-7)

**What:** Convert a voice note into text, and know how accurate it is.

**How:**

- Fetch the media URL from the Meta API, then download the OGG/Opus audio with the access token.
- Convert with ffmpeg through os/exec if a provider needs a different format.
- Define an ASR interface: Transcribe(ctx, audio) (Result, error), where Result has text, language and confidence. Implement Sarvam, Whisper and optionally Google STT behind it.
- Record 30-50 voice notes from your parents (different rooms, speeds, background noise). Write correct transcripts by hand.
- Compute word error rate with a small Python jiwer script or your own Go edit-distance function. Compare providers and pick the best.
- If confidence is low, reply asking them to repeat instead of guessing.
- Delete audio right after processing and keep only the text.

**Done when:** You have a benchmark table (provider, WER, latency, cost) and a chosen default.

## Phase 4: Data layer (Weeks 6-8, alongside Phase 3)

**What:** Reliable storage.

**How:**

- Create the schema from section 5 using goose migrations.
- Write your queries in SQL files and generate Go code with sqlc.
- Use transactions for anything that changes orders and audit_log together.
- Add database constraints (CHECK on status values, foreign keys, unique keys).
- Tests use testcontainers-go to run a real Postgres, not mocks.

**Done when:** You can create a user, saved places and an order, and illegal state transitions are rejected.

## Phase 5: Conversation engine (Weeks 8-11)

**What:** The brain. A state machine is a fixed set of steps, so the bot never improvises with money.

**States (cab flow):** idle, awaiting_pickup, awaiting_drop, awaiting_confirmation, booking, active_ride, completed, cancelled, error.

**How:**

- Implement states as Go constants and transitions as a table (map of state and event to next state). A hand-written machine teaches more than a library; looplab/fsm is an option later.
- Call Claude through anthropic-sdk-go with tool use. Define tools such as extract_ride_request returning pickup, drop and time. The model returns structured data, never free-form decisions.
- Validate every LLM output with go-playground/validator. Reject or re-ask on anything unexpected.
- Resolve ghar, mandir and similar words from saved_places. If a place is unknown, ask a short question.
- Always send a plain confirmation (pickup, drop, estimated fare) and require an explicit haan before booking.
- Handle changes of mind, cancellations, silence timeouts, and unrelated messages.
- Replies are short, polite and in the parent's language. Keep a prompt file per language.

**Tests:** Table-driven tests for each transition, plus a set of 30+ scripted conversations (wrong place, changed mind, abuse, gibberish, wrong language).

**Done when:** A voice note reaches the confirm step with correct data, and bad inputs never skip the confirmation.

## Phase 6: Safety layer (Weeks 11-12)

**What:** Rules in code that the AI cannot override. Elderly users are prime scam targets, so this is the most important phase.

**How:**

- A safety package called before every booking. It checks the per-ride cap, daily cap, allowed hours and known places.
- Unusual requests (a far destination, rapid repeated bookings, a new place) require approval from the child over WhatsApp before proceeding.
- Treat forwarded messages, links and pasted text as untrusted. Never let message content change the system prompt or tools (prompt-injection defence).
- Rate limiting per user with httprate or redis_rate.

- A per-user kill switch the child can trigger from the dashboard or by replying to an alert.
- Safe failure: when unsure, ask again or escalate to the child. Never guess an address.

**Done when:** Tests prove that each rule blocks a booking even when the LLM output asks for it.

## Phase 7: Child dashboard (Weeks 12-14)

**What:** A web app where the child sets things up and monitors.

**How:**

- Next.js with TypeScript, Tailwind and shadcn/ui.
- Go API endpoints with chi, protected by JWT (golang-jwt) after phone OTP login.
- Pages: link a parent (verification code), saved places (with a map picker), spend limits, order history, alerts, and a kill switch.
- Write an OpenAPI spec and generate a typed client so the frontend and backend stay in sync.
- Validate all input on the server, add CORS rules and rate limits.

**Done when:** A child can link a parent, add Ghar and Mandir, set limits, and see rides appear in history.

## Phase 8: Booking integration (Weeks 14-16)

**What:** Making the ride actually happen.

**How:**

- Define a Provider interface: Search, Book, Cancel, Status.
- **Step 1, deep-link provider:** Send a pre-filled Uber, Ola or Rapido link with pickup and drop. The parent taps once and confirms in that app. It works on day one.
- **Step 2, Beckn/ONDC sandbox provider:** Learn the flow (search, select, init, confirm, status, cancel). Beckn is JSON over HTTP, so build the structs and calls in Go. Study the open Namma Yatri code (mostly Haskell) to understand how a Beckn app behaves, but do not copy it.
- Check which cab providers are live on ONDC in your city before investing time.
- Test each provider against httptest fake servers so tests never hit real services.

**Done when:** A confirmed request produces either a working deep link or a sandbox booking, and the order state updates correctly.

## Phase 9: Reliability (Weeks 16-17)

**What:** Handle failures gracefully.

**How:**

- asynq retries with exponential backoff, timeouts, and scheduled tasks for ride-status polling.
- context deadlines on every external call.
- A circuit breaker (sony/gobreaker) around ASR, LLM and provider calls.
- Fallback messages, for example: kuch gadbad ho gayi, main aapke bete ko bata deta hoon. Also notify the child.
- Graceful shutdown so in-flight jobs finish during deploys.
- Reconciliation job that finds orders stuck in booking and fixes or flags them.

**Done when:** Killing Redis, Postgres or a provider mid-flow leads to a clean retry or a clear message, never a duplicate or lost booking.

## Phase 10: Observability and security (Weeks 17-18)

**What:** Know when things break and close the obvious holes.

**How:**

- sentry-go for errors, Prometheus metrics, Grafana Cloud dashboards, UptimeRobot for uptime.
- Track: booking success rate, ASR failure rate, LLM latency, p95 response time, queue depth, safety-block count.
- Redact phone numbers and addresses in logs. Add request IDs to trace one message through all services.
- Run govulncheck, gosec and Dependabot in CI.
- Load test with k6 or vegeta and fix what breaks.
- Secrets only in environment variables or the platform's secret store. Rotate any key that ever touches Git.
- Write a short threat model: what can a malicious message, a stolen phone or a leaked token do?

**Done when:** You can answer: Is it up? How many bookings failed today? Why?

### Phase 11: Deploy and real-world test (Weeks 18-20)

**What:** Run it for real.

**How:**

- Multi-stage Dockerfile producing a small image with one static Go binary.
- Deploy to Fly.io or Railway with separate staging and production environments, Cloudflare in front, and CI/CD that deploys on merge to main.
- Run it with your family for at least two weeks. Ask them to use it naturally. Log every failure and misunderstanding, and fix the top ones.
- Be open with your family about what data is stored. Keep audio deletion and a delete-my-data command working (good DPDP-style habits).
- Finish with a README, architecture diagram, a 2-minute demo video, and the polished decisions log.

**Done when:** A parent books a real ride by voice note without help.

## 7. Milestones

| By week | Milestone |
|---|---|
| 2 | Comfortable writing Go; mini project done |
| 5 | Verified, deduplicated WhatsApp webhook running |
| 8 | Voice note becomes a transcript stored in Postgres; ASR benchmark done |
| 12 | Conversation reaches confirm, and safety rules can block it |
| 14 | Dashboard works end to end |
| 16 | Real deep link or sandbox booking created |
| 18 | Reliability, monitoring and security pass done |
| 20 | Family uses it for real; README and demo ready |

## 8. Testing strategy

- **Unit tests:** Table-driven tests for the state machine, safety rules and parsers.
- **Integration tests:** testcontainers-go with real Postgres and Redis.
- **Contract tests:** httptest fake servers for WhatsApp, ASR, Claude and booking providers.
- **Conversation tests:** A library of scripted dialogues run in CI, including adversarial ones.
- **ASR benchmark:** Track word error rate over time as a number in your repo.
- **Load and failure tests:** k6 plus deliberately killing dependencies.
- **Coverage goal:** High coverage on safety, conversation and order code; do not chase 100 percent everywhere.

## 9. Risks and how to handle them

| Risk | Mitigation |
|---|---|
| Poor ASR on parents' accent | Benchmark early (Phase 3), try multiple providers, ask to repeat on low confidence |
| No open booking API | Deep links first, ONDC sandbox second, provider interface keeps options open |
| WhatsApp policy or pricing changes | Keep the WhatsApp layer isolated in its own package; read the current policy before building |
| LLM misunderstands and books wrong ride | Explicit confirmation, validation, safety layer, state machine |
| Scope creep | Cab only for v1; food and movies are later versions |
| Burnout on a solo project | Keep weekly milestones small, demo something each week |
| Privacy | Delete audio, redact logs, minimal data, clear family consent |

## 10. Weekly habits

- Pass context.Context as the first argument of functions that do I/O.
- Wrap errors with fmt.Errorf and %w so you keep the cause.
- Define interfaces where they are used, not where they are implemented.
- Keep packages small and focused; avoid global state.
- Write the test first for state machine and safety logic.
- Update docs/decisions.md every time you choose or change something.
- Commit small, merge often, and keep CI green.

## 11. After v1 (future ideas)

- Food ordering through ONDC providers
- A call-my-son escalation and emergency handling
- Voice replies (text-to-speech) so parents hear answers
- More languages and regional accents
- Spend alerts and weekly summaries for the child
- Proper business verification and a public launch
- A subscription model paid by the child

## 12. Starter checklist (this week)

1. Decide your parents' language and city.
2. Set up Go, VS Code with the Go extension, Docker and Git.
3. Finish Tour of Go and Go by Example.
4. Create the Meta developer account and WhatsApp test number.
5. Create the GitHub repo with CI and the decisions log.
6. Start recording voice notes from your parents for the benchmark.